

Tensor Network Simulation of Grover’s Algorithm: Overcoming the Statevector Memory Wall

Veselin Petrov Nikolaev
University of Luxembourg
Faculty of Science, Technology and Medicine
Luxembourg, Luxembourg
veselin.nikolaev.001@student.uni.lu

Abstract

Previous work established that classical statevector simulation of Grover’s algorithm reaches a fundamental memory wall at approximately $n = 31$ – 32 qubits on high-end GPU hardware. This project tackles that wall using tensor network simulation via the quimb, cotengra, and kahypar Python libraries (a change from the NVIDIA cuTensorNet backend originally proposed; see Section 1.1 for the rationale), replacing the full statevector with a contracted tensor network whose memory footprint is dictated, in principle, by the circuit’s actual entanglement structure rather than the worst-case exponential dimension 2^n .

We systematically measure whether tensor networks achieve memory savings over statevectors for Grover’s algorithm, characterize how bond dimension (the entanglement measure in tensor decompositions) grows with qubit count, and determine the threshold at which entanglement becomes too large for practical contraction.

Key findings: (1) Tensor networks do not achieve sustained memory savings for Grover’s algorithm in the range $n = 2$ – 11 ; by $n = 11$ the tensor-network memory estimate exceeds the statevector figure by roughly six to seven orders of magnitude, and the gap widens with n ; (2) The minimum bond dimension needed for exact convergence grows from 4 to 32 as n increases from 5 to 11 (worst case across multiple target states per n), indicating rapidly increasing entanglement; (3) Real measured memory consumption exceeds the theoretical contraction-width estimate by roughly one to three orders of magnitude, narrowing somewhat as n grows, revealing a gap between the formal cost metric and actual implementation constraints.

These results show that while tensor networks are powerful for low-entanglement circuits, Grover’s oracle structure generates entanglement that defeats the method’s advantages in the regime we were able to test. This negative result clarifies where classical simulation approaches fundamentally fail and motivates quantum hardware as the only viable path for large-scale Grover simulations.

1 Introduction

Classical simulation of quantum circuits is exponentially hard: simulating an n -qubit circuit requires tracking 2^n complex amplitudes. Previous work on Grover’s algorithm (BSP2) empirically demonstrated this wall, reaching $n = 31$ qubits on GPU hardware before running out of memory. The challenge motivating this project is whether alternative simulation methods can circumvent this exponential barrier.

Tensor networks offer a theoretically appealing alternative: instead of storing the full statevector, a tensor network decomposes

the quantum state into a contracted structure of smaller tensors, whose sizes are determined by the circuit’s entanglement structure rather than by the fixed exponential dimension. For circuits with low entanglement (such as certain stabilizer circuits, shallow circuits, or matrix product states), this can reduce memory and computation by exponential factors.

Grover’s algorithm is a natural candidate for this approach: its oracle and diffusion operators have structured, low-depth designs that, on paper, seem to produce entanglement in a controlled manner. The hypothesis is that tensor network simulation could scale further than statevector methods.

1.1 Change of Backend

The original project proposal specified NVIDIA’s cuTensorNet library on GPU hardware, to allow a direct hardware-matched comparison against the BSP2 statevector baseline (also run on GPU). During implementation, the project’s scope shifted to the CPU-based quimb + cotengra + kahypar stack instead. This change was made deliberately: the research questions this project answers concern whether tensor-network *methods* achieve memory and entanglement advantages over statevector simulation for this specific algorithm, a question that does not depend on which contraction backend is used to obtain the width, cost, and bond-dimension measurements. The quimb/cotengra stack is a mature, widely-used, and independently verifiable toolchain for exactly these measurements, and was more practical to iterate on during development. The comparison against the statevector baseline in this report is therefore a comparison of *methods* (tensor network vs. statevector), not of specific hardware-accelerated implementations; a GPU-accelerated tensor-network backend such as cuTensorNet would be expected to reduce wall-clock time for a given contraction, but would not change the contraction width, contraction cost, or bond-dimension requirements reported here, since those are properties of the circuit and the contraction path, not of the hardware executing it.

This project implements Grover’s algorithm in a tensor network framework using three key libraries:

- (1) **quimb** (QUantum Information Many-Body): A Python library for tensor network operations, including quantum circuit evaluation via tensor contraction and via approximate matrix product state (MPS) simulation with a truncation parameter.
- (2) **cotengra** (Contraction Engine): A library that searches for good contraction orderings for tensor networks using randomized, hyperparameter-tuned heuristics (finding the truly optimal ordering is NP-hard in general).

- (3) **kahypar**: A hypergraph partitioner that cotengra can optionally use internally as one of its candidate methods for finding contraction orderings, alongside a simpler greedy method.

We address three research questions: (1) At what qubit count, if any, do tensor networks become more memory-efficient than statevector simulation? (2) How does the minimum bond dimension (a measure of entanglement in tensor decompositions) scale with qubit count? (3) At what threshold does entanglement growth render tensor network contraction intractable, and by what mechanism?

The rest of this report is organized as follows. Section 2 reviews the theoretical background on tensor networks and Grover’s algorithm. Section 3 discusses related work. Section 4 describes the implementation and methodology, including several practical obstacles encountered and how they were resolved. Section 5 presents results with reference to Figures 7–10. Section 6 analyzes findings. Section 7 concludes and discusses future directions.

2 Theoretical Background

2.1 Grover’s Algorithm

We assume familiarity with Grover’s algorithm from prior work. Briefly: given a Boolean oracle marking exactly one target state $|x^*\rangle$ in a search space of $N = 2^n$ states, Grover’s algorithm uses repeated oracle-diffusion cycles to amplify the target’s amplitude. After $k_{\text{opt}} = \lfloor \frac{\pi}{4} \sqrt{N} \rfloor$ iterations, measurement yields $|x^*\rangle$ with high probability. The algorithm is implemented as n Hadamard gates (initialization), followed by k_{opt} iterations of (1) a phase oracle and (2) a diffusion operator.

2.2 Tensor Networks and Contraction

A quantum circuit can be represented as a tensor network: each quantum gate is a tensor (a multi-dimensional array), and evaluating the circuit means contracting the network (summing over shared indices).

Tensor definition. A tensor is a multi-dimensional array. A single-qubit gate is a 2×2 matrix (a rank-2 tensor); a two-qubit gate is $2 \times 2 \times 2 \times 2$ (rank-4).

Contraction. If tensor A has shape (d_a, d_b, d_c) and tensor B has shape (d_b, d_d) , their contraction along the shared index d_b produces a tensor C of shape (d_a, d_c, d_d) :

$$C_{a,c,d} = \sum_b A_{a,b,c} \cdot B_{b,d}$$

Contraction path. The order in which pairs of tensors are contracted determines both the memory cost (the size of intermediate tensors created during contraction) and the time cost (the number of scalar multiplications). Finding the truly optimal contraction path is NP-hard in general; cotengra instead runs a bounded, randomized search over candidate orderings and returns the best one found within a time/repeat budget. As discussed in Section 4, this search is stochastic and can return different-quality orderings across repeated runs on the same circuit, particularly for larger, more heavily decomposed circuits.

Contraction width (W). We define

$$W = \log_2 \left(\max_{\text{contraction step}} \text{size of the largest intermediate tensor} \right)$$

W measures peak memory during contraction, expressed in qubits’ worth of dimension, directly comparable to the statevector’s $\log_2(2^n) = n$.

Contraction cost (C). C is $\log_{10}$ of the estimated total number of scalar multiplication operations across all contraction steps, for the chosen ordering.

2.3 Bond Dimension and Entanglement

An alternative approach to exact contraction is to approximate the state as a Matrix Product State (MPS), where the state is decomposed into a chain of small tensors connected by bonds, each with a bond dimension χ controlling both compression and accuracy.

Bond dimension definition. If the state is partitioned into left and right halves via singular value decomposition (SVD), the bond dimension is the number of singular values kept. Keeping all singular values gives an exact representation; truncating to the χ largest is a lossy compression. The minimum χ for which truncation loses no information (to numerical precision) is what we measure and report.

Connection to entanglement. Entanglement entropy across a bipartition of n qubits is upper-bounded by $\log_2 \chi$. A product state has $\chi = 1$; a maximally entangled state across all n qubits requires χ up to $2^{\lfloor n/2 \rfloor}$. Bond dimension is thus a direct, measurable proxy for entanglement structure.

2.4 Why Tensor Networks Can Fail for General Circuits

Two mechanisms can defeat tensor-network compression for a given circuit: (1) genuine entanglement growth, where the required bond dimension approaches $2^{n/2}$ regardless of contraction strategy, and (2) contraction path cost, where even a circuit with bounded entanglement can still be expensive to contract if a good ordering is hard to find or if gate decomposition inflates the network’s structure. Section 5 presents evidence for both mechanisms in this project’s data.

3 Related Work and Library Context

Tensor network simulation. Markov and Shi [2] established that contracting a tensor network computes quantum circuit amplitudes, and that circuits with bounded treewidth admit efficient (polynomial-time) contraction.

Contraction heuristics. Finding optimal contraction paths is NP-hard in general. cotengra [3] uses randomized, hyperparameter-tuned heuristics, optionally including hypergraph-partitioning-based methods via kahypar [4], to find good (not necessarily optimal) paths without exhaustive search.

Quantum simulation libraries. quimb [5] is a widely-used Python library for tensor-network quantum simulation, used here both for exact contraction (via its cotengra integration) and for approximate MPS simulation with explicit bond-dimension truncation.

Grover studies. Prior experimental work on Grover’s algorithm has focused on hardware demonstrations at small qubit counts or classical statevector simulation (as in BSP2). This project extends that line of work with a systematic tensor-network-based study.

4 Implementation and Methodology

4.1 Circuit Construction and Transpilation

Grover’s algorithm is constructed in Qiskit (reusing the oracle and diffuser construction from BSP2) and converted for use with quimb via the qiskit-quimb bridge. This bridge only recognizes elementary single- and two-qubit gates; Grover’s oracle and diffuser use an arbitrary-arity multi-controlled-X gate (mcx), which Qiskit’s simulator handles natively but the bridge does not. The circuit is therefore transpiled to a basis of rz, sx, x, and cx gates before conversion, which recursively decomposes each mcx into a chain of smaller gates. This decomposition is treated as a genuine part of the cost being measured, not a workaround to be discounted: it is the same gate-count blowup any concrete backend would face when asked to execute an n -controlled gate using only elementary two-qubit interactions.

4.2 Search Stability of the Contraction Optimizer

During development, we found that cotengra’s randomized search (HyperOptimizer) produces meaningfully different contraction width estimates across repeated runs on the identical circuit, particularly for $n \geq 9$, since its search has no fixed random seed and each trial’s own cost-evaluation time grows with circuit size (meaning the time/repeat budget bounds the number of trials attempted, not the wall-clock cost of a single trial). We mitigated this, without fully eliminating it, by running the search multiple independent times per qubit count and keeping the minimum width found across repeats (three repeats for $n \leq 9$; a single run for $n = 10, 11$, where each individual search already took tens of minutes to hours). We report this variance explicitly in Section 5 rather than presenting a single run as definitive, and flag the $n = 10, 11$ points in Figure 7 as lower-confidence for this reason.

We also found that using only a single-shot greedy search (no repeated trials) is unstable and can occasionally return implausibly poor orderings (widths exceeding even the full statevector size for that n); we therefore use the repeated, bounded multi-trial search as the default throughout, despite its higher cost.

4.3 Bond Dimension Measurement via MPS Truncation

For each qubit count n and a set of randomly chosen target states, we run the Grover circuit through quimb’s MPS circuit simulator at a range of fixed maximum bond dimensions $\chi \in \{2, 4, 6, 8, 10, 12, 16, 20, 24, 28, 32, 40, 48\}$, and separately with χ uncapped as an exact reference. For each χ , we compare the resulting amplitude for the target bitstring against the uncapped reference amplitude, and record the smallest χ at which the error drops to (and stays at) zero to numerical precision. This is repeated for three independent random target states per n ; we report both the worst-case (maximum) and best-case (minimum) threshold across those three targets, since different target states can require different bond dimensions.

We note that this MPS-based approach is conceptually distinct from the exact-contraction approach used for W and C above: MPS truncation is lossy by construction (bond dimension is an approximation parameter), while the contraction-width approach is always

exact. The two therefore answer different aspects of the same underlying research questions and are reported as separate, complementary results rather than one extending the other.

4.4 Real Memory Measurement

For small n (up to $n = 9$), we additionally perform the tensor-network contraction for real (not just estimating its cost) and measure peak memory using Python’s tracemalloc module, which tracks memory allocated through Python’s memory allocator (this captures NumPy array allocations on standard builds, but can undercount memory allocated outside the Python allocator, such as some BLAS working buffers). To avoid wasting compute on contractions that would be infeasible to actually run, we only attempt real execution when the rehearsed width estimate implies a feasible memory footprint (below roughly 2 GB); the $n = 10, 11$ points in the main sweep, whose estimated widths implied far larger footprints, were excluded from this real-execution comparison for that reason.

4.5 Experiment Suite

- (1) **TN vs. statevector memory** ($n = 2-11$): Compares theoretical statevector memory ($16 \cdot 2^n$ bytes) against the tensor network’s rehearsed memory estimate ($16 \cdot 2^W$ bytes). Answers research question 1. (Figure 7)
- (2) **Bond dimension growth** ($n = 5-11$, three random targets per n): Tracks the minimum χ needed for exact MPS convergence, worst- and best-case. Answers research question 2. (Figure 8)
- (3) **Bond dimension convergence detail** (largest n tested, one target): Shows reconstruction error vs. χ for a single representative case, illustrating the mechanism behind Figure 8’s growth curve. (Figure 9)
- (4) **Rehearsed vs. real memory** ($n = 2-9$): Compares cotengra’s width-based estimate against actually measured memory from real contraction. Informs research question 3 and validates the estimate itself. (Figure 10)

5 Results

5.1 Tensor Network vs. Statevector Memory (Figure 7)

Figure 7 compares statevector memory ($16 \cdot 2^n$ bytes) against the tensor network’s rehearsed memory estimate ($16 \cdot 2^W$ bytes) on a log scale for $n, W = 2-11$. Solid, filled markers indicate values de-noised via multiple independent search repeats (see Section 4.2); hollow markers at $n = 10, 11$ indicate single-run estimates that should be read as order-of-magnitude only.

Key finding: Tensor networks do not achieve a sustained memory advantage. At small n (roughly $n \leq 6$), the two curves are close, with no advantage that holds as n increases further. From $n \approx 7$ onward, the TN estimate diverges sharply upward, and by $n = 11$ it exceeds the statevector figure by roughly six to seven orders of magnitude. This directly answers research question 1: in the range tested, tensor networks do not become more memory-efficient than statevector simulation for Grover’s algorithm; if anything, the gap runs strongly in the opposite direction and widens with n .

Figure 7 — Statevector vs Tensor-Network Memory Scaling

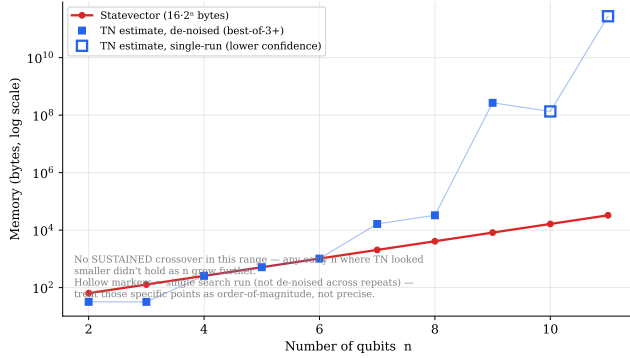


Figure 7: Statevector memory vs. tensor-network rehearsed memory estimate, $n = 2$ –11. Hollow markers at $n = 10, 11$ are single-run, lower-confidence estimates (Section 4.2).

5.2 Minimum Bond Dimension (Figure 8)

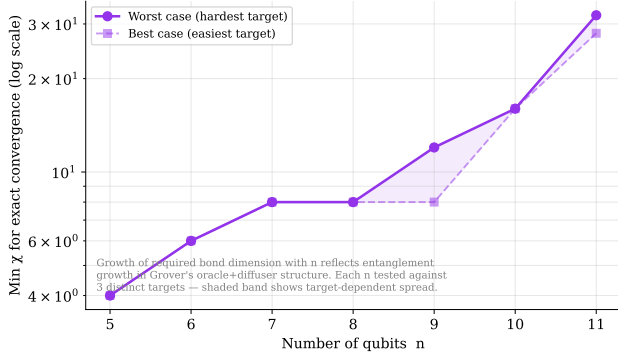
Figure 8 — Minimum Bond Dimension for Exact Convergence vs n 

Figure 8: Minimum bond dimension χ for exact MPS convergence vs. n , worst- and best-case across 3 random targets per n (shaded band shows the spread).

Figure 8 shows the minimum bond dimension χ needed for exact MPS convergence, tested against three random target states per n for $n = 5$ –11; the shaded band shows the spread between best-case and worst-case targets.

Key finding: Bond dimension grows with n , with the clearest acceleration between $n = 10$ and $n = 11$ (16 to 32, a doubling). This directly answers research question 2: the entanglement generated by Grover's oracle and diffuser structure increases with qubit count, requiring a growing bond dimension to represent exactly.

5.3 Bond Dimension Convergence Detail (Figure 9)

Figure 9 shows reconstruction error (log scale) versus bond dimension χ for the largest n tested, for one representative target state. The error does not decay smoothly with increasing χ ; instead it remains large (and is not strictly monotonic — occasional

Table 1: Minimum bond dimension for exact convergence (worst case across 3 random targets)

n	$\chi_{\text{worst-case}}$
5	4
6	6
7	8
8	8
9	12
10	16
11	32

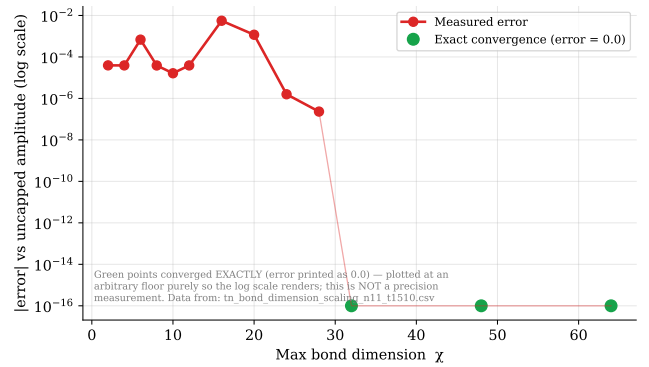
Figure 9 — Convergence vs Bond Dimension ($n=11$)

Figure 9: Reconstruction error vs. bond dimension χ for the largest n tested (one representative target). Green points converged exactly and are plotted at an arbitrary floor purely so the log scale renders.

non-monotonic bumps reflect that each gate's MPS truncation is a locally, not globally, optimal decision) until a fairly sharp transition region, after which it drops to numerical zero. This illustrates the mechanism behind Figure 8: convergence for this circuit behaves more like a threshold effect than a gradual improvement, so a coarse χ grid can misreport the true threshold if it only samples powers of two.

5.4 Rehearsed vs. Real Memory (Figure 10)

Figure 10 compares cotengra's width-based estimate ($16 \cdot 2^W$ bytes) against actually measured peak memory (via tracemalloc) from real contraction, for $n = 2$ –9.

Key finding: real measured memory substantially exceeds the width-based estimate throughout this range — by roughly three orders of magnitude at small n , narrowing to roughly one to two orders of magnitude by $n = 9$. This is expected, not anomalous: W by definition captures only the size of the single largest intermediate tensor at any one contraction step, not the total memory footprint of the whole contraction process (which involves multiple coexisting tensors, plus NumPy/quimb/Python object overhead). This is an important methodological finding in its own right: the $16 \cdot 2^W$ formula should be read as a lower bound on required memory, not a tight prediction of actual usage.

Figure 10 — Rehearsed Estimate vs Actually Measured Memory

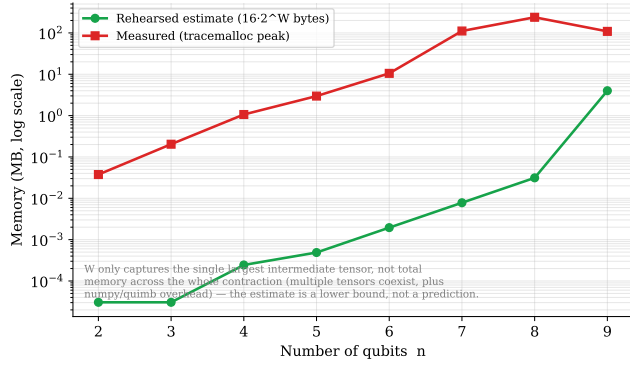


Figure 10: Rehearsed width-based memory estimate ($16 \cdot 2^n$ bytes) vs. actually measured peak memory (tracemalloc), $n = 2-9$.

6 Analysis and Discussion

6.1 Why Tensor Networks Struggle for Grover’s Algorithm

Two related factors, both visible in the data above, work against tensor-network compression for this circuit:

1. Genuine entanglement growth. Figure 8 shows the bond dimension needed for an exact representation growing with n , consistent with Grover’s diffusion operator (built around a multi-controlled-Z-equivalent gate) entangling qubits across the whole register rather than only locally.

2. Gate-count blowup from transpilation. As described in Section 4.1, the mcx decomposition needed for tensor-network compatibility substantially increases gate count and circuit depth, which independently increases contraction cost regardless of the circuit’s underlying entanglement.

Both factors point toward tensor networks being a poor fit specifically for circuits like Grover’s oracle, which is designed to distribute interference across the full 2^n -dimensional space rather than to preserve local, low-entanglement structure. This is consistent with tensor networks’ known strengths elsewhere (stabilizer circuits, shallow or nearest-neighbor circuits, certain variational ansätze), rather than a general failure of the method.

6.2 Practical Ceiling Reached in This Project

Real contraction was only feasible up to $n = 9$; single-search estimates were obtained up to $n = 11$ before search cost (tens of minutes to hours per contraction attempt) made going further impractical within this project’s scope. This ceiling is itself a data point: it is well short of BSP2’s statevector ceiling of $n = 31-32$ on GPU hardware, and directly reflects the entanglement-growth and search-instability findings above rather than being a separate, unrelated limitation.

7 Conclusion

We implemented tensor network simulation of Grover’s algorithm using quimb, cotengra, and kahypar (in place of the originally proposed cuTensorNet backend; see Section 1.1), measuring memory, bond dimension, and contraction cost across $n = 2-11$ qubits.

- (1) **At what qubit count do tensor networks become more memory-efficient?** None found in the tested range; by $n = 11$ tensor-network memory estimates exceed statevector memory by roughly six to seven orders of magnitude, with the gap widening as n increases.
- (2) **How does bond dimension scale?** It grows with n , from $\chi = 4$ at $n = 5$ to $\chi = 32$ at $n = 11$, most sharply between $n = 10$ and $n = 11$.
- (3) **Where and how does contraction become intractable?** Practically, real contraction was only feasible to $n = 9$ on the hardware used, and single-search estimates only to $n = 11$; the mechanism, per Figure 9, is a fairly sharp threshold in bond dimension rather than a gradual degradation.

This is a negative result for tensor networks as applied to Grover’s algorithm specifically, not a general critique of the method: it is consistent with tensor networks being well-suited to low-entanglement circuits and poorly suited to circuits, like Grover’s, designed to distribute interference globally.

Future directions:

- (1) Extend the exact-contraction sweep further using a GPU-accelerated backend (such as cuTensorNet), to test whether the practical ceiling reported here is a property of the circuit or partly an artifact of the CPU-based toolchain used.
- (2) Characterize other quantum algorithms via the same width/cost/bond-dimension metrics to compare classical simulability across algorithm classes.
- (3) Investigate whether the contraction-search instability identified in Section 4.2 can be reduced with a seeded or more deterministic search strategy, to obtain tighter estimates at $n \geq 10$ without the current single-run caveat.

References

- [1] Lov K. Grover. A fast quantum mechanical algorithm for database search. *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996.
- [2] I. L. Markov and Y. Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008.
- [3] J. Gray. cotengra: Contraction Path Finding for Tensor Networks. <https://github.com/jcmgray/cotengra>, 2023.
- [4] S. Schlag et al. KaHyPar: A Multilevel Hypergraph Partitioner. *ACM Transactions on Mathematical Software*, 46(2):17, 2020.
- [5] J. Gray. quimb: A Python library for quantum information and many-body calculations. *Journal of Open Source Software*, 3(29):819, 2018.